Make Observable Outfitters observable

You're a Forward Deployed Engineer embedded with the team behind Observable Outfitters, a small online store. The code is in the `app/` folder that came with this brief. The store works, but the team can't see anything: the product manager can't say where shoppers drop off, on-call hears about problems from customers rather than alerts, and nobody knows how slow checkout is or how often payments fail. The team uses Grafana Cloud and wants to try Aiden for SRE, StackGen's AI SRE agent, to turn that data into action. Your job is to get them there.

// Before you start

You'll need Node.js 20+ and npm, Git, Google Chrome *(or run `npx playwright install chromium`)*, a Grafana Cloud account *(the free tier is enough)*, an Aiden for SRE Community Edition account *(also free)*, and the AI coding assistant of your choice. Docker is only needed if your pipeline uses it. Check that the app runs:

```
cd app && npm install
npm test        # 7 end-to-end tests, should all pass
npm run dev     # http://localhost:3000, walk through a purchase
```

`app/README.md` describes the user journey, configuration and the traffic simulator.

// What to do

1. Instrument the application. Add telemetry that answers real questions about reliability *and* the product. For example: where do people drop off? Why does checkout fail? Is the site fast for real users? Can we get from "a customer complained" to the root cause? Traces, metrics, logs, real-user monitoring, business events: what you capture and how is up to you. Don't change how the product behaves *(if you find bugs, let your telemetry show them and write them up)*. Keep `npm test`, `npm run lint` and `npm run typecheck` passing, and make sure the app still works if Grafana Cloud can't be reached.
2. Send the data to Grafana Cloud. Use any pipeline: OTLP straight from the app, Grafana Alloy, an OpenTelemetry Collector, Faro, or something else. Be ready to explain your choice. Generate traffic with `npm run simulate`, then show the data answering at least one real question with a dashboard, a saved query, an alert or whatever works best. Never commit credentials.
3. Build two Aiden workflows that help the team improve the product. Sign up for Aiden for SRE Community Edition at ai.stackgen.com, connect your Grafana Cloud stack and build two workflows. There's no approved list; we want to see what you think is worth automating. A good workflow helps a specific person on the team, leads to a decision or action rather than a report, depends on the telemetry you chose in step 1, and has actually run. For each one, document the trigger, the steps, the output *(with a real example)* and what you'd improve.

// What to send us

- A link to your Git repo with your instrumented `app/`. Commit as you go; we read the history. If the repo is private, give us access.
- `WRITEUP.md` in the repo root, two pages at most, covering:

    - architecture: how telemetry gets from the browser and server to Grafana Cloud *(a diagram helps)*;
    - decisions and trade-offs, including what you chose *not* to instrument;
    - what the data told you about the product;
    - your two Aiden workflows, with screenshots or exports;
    - how you used AI: where it helped, where it was wrong, and how you noticed;
    - what you'd do with another week.
- Setup steps in the repo README: the environment variables and commands we need to run your version against our own Grafana Cloud stack.
- A 1–2 minute screen recording *(Loom, YouTube unlisted or a file link)* showing your telemetry in Grafana Cloud and both Aiden workflows running. Link it from `WRITEUP.md`.

In a follow-up session *(about 45 minutes)* you'll demo your work and we'll ask about your choices.

// Expectations

- Use AI. We mean it. FDEs at StackGen use AI tools heavily. We aren't testing whether you can write OpenTelemetry boilerplate from memory; we're testing your judgement: what you ask for, what you accept and what you push back on.
- There's no single right answer. We care more about *why* you chose your approach than which one you picked.
- Show us how you think. Write down your reasoning, dead ends and assumptions. "I tried X, it didn't work because Y, so I did Z" is worth more than a polished result with no explanation.
- Scope it yourself. Expect to spend about 4 hours. If you run out of time, stop and write down what's left and how you'd do it. Small, working and well explained beats big and half-working.
- Think like an FDE. If you handed this to the team tomorrow, would they understand it, trust it and be able to keep it running?

// How we'll evaluate

Instrumentation that answers real questions, with front end and back end connected. A pipeline that reliably reaches Grafana Cloud, handles secrets properly and is easy for someone else to run. Aiden workflows that lead to action and depend on your telemetry. Insight about the store and its customers drawn from the data. Clear trade-offs and honest limits in your write-up and demo. Effective, critical use of AI.

// Questions?

If something is unclear, make a reasonable assumption, write it down and keep going. If you're truly blocked *(an account problem, say)*, email your recruiting contact. Good luck, and have fun with it.